**Institut Teknologi Bandung**

Directorate of Continuing Professional Education

in partnership with

CHAPTER 12

# Object Detection

Boxes and labels, predicted together — a simplified YOLO built from scratch, and a pretrained detector that generalises to a pointillist painting.

**Duration** 2.5 hours

**Instructor** Rahman Indra Kesuma, S.Kom., M.Cs.

**Source** Chollet & Watson, Deep Learning with Python 3e — chapter 12

LEARNING OUTCOMES

# Session Objectives

By the end of this session, participants can:

1. Say when to use **detection rather than segmentation**, on both compute and labelling grounds.
2. Distinguish **two-stage (R-CNN)** from **single-stage (YOLO, RetinaNet, SSD)** detectors, and name the trade-off.
3. Build a **YOLO detection head** on a pretrained backbone, and read the shape of its grid output.
4. Explain why the **confidence target is an IoU score** rather than a 0/1 flag.
5. Run a **pretrained RetinaNet**, and explain why it handles small and large objects better.
6. Say why single-stage detectors are **more robust to unfamiliar inputs** than two-stage ones.

BOOK
Chapter 12 — full text

NOTEBOOK
01_coco_and_boxes.ipynb

NOTEBOOK
02_yolo_from_scratch.ipynb

NOTEBOOK
03_pretrained_retinanet.ipynb

REPO
Author's official notebook

# What detection is for

**Counting**
How many instances of an object are in this image.

**Tracking**
Run detection on every frame to follow objects over time.

**Cropping**
Cut out the region containing an object and send a higher-resolution patch to a classifier or an OCR model.

# If segmentation is a superset, why detect at all?

**Computational cost**
A good detector typically runs **much faster** than a segmentation model.

**Labelling cost**
Segmentation needs **pixel-precise masks**, far more time-consuming to produce than bounding boxes.

So the rule: **always use a detector if you do not need pixel-level information** — for instance if all you want is **to count things**

# 01

# Two-stage vs single-stage

Two broad families, and why one of them won.

# Two-stage: propose, then classify

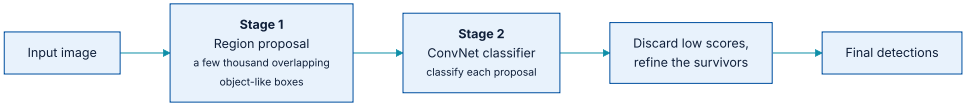



Figure 12.2 — an R-CNN extracts region proposals, then classifies each one with a ConvNet.

The first stage is **not very smart**: it produces boxes around areas that merely *look object-like*. The second stage decides what, if anything, is in each one.

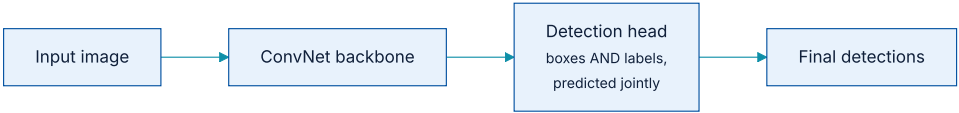
# Why that is expensive

> It works well, but it requires classifying **thousands of patches for every single image**. That rules it out for most real-time applications and for embedded systems.

The book's practical take is blunt: you rarely need R-CNN at all. **With a large server GPU you are usually better off with a segmentation model** like SAM; **if you are resource-constrained you want a single-stage detector**. **Neither case points at a two-stage model.**

SECTION 12.1.2

# Single-stage: predict both at once




One model jointly predicts box coordinates and labels.

The main families are **RetinaNet**, **SSD** (Single Shot MultiBox Detector), and the **YOLO** family — *You Only Look Once*, named after the meme on purpose.

SECTION 12.1.2

# The trade-off, and the state of play

- Single-stage detectors are **significantly faster and more efficient** than two-stage ones, with a **minor potential trade-off in accuracy**.
- **YOLO is arguably the most popular object detection model there is**, especially for real-time applications.
- A new version appears roughly every year — and interestingly, **each new version tends to come from a different organisation**.

There were twelve YOLO versions at the time of writing. This chapter recreates **the original from 2015**, which is simpler to work with.

# 02

# Training a YOLO model from scratch

COCO, a grid, and a loss with an unusual target.

SECTION 12.2

# An honest warning before starting

> Building a detector is **an undertaking** — not because anything in it is theoretically complex, but because there is **a lot of code just to manipulate bounding boxes and predicted output**

The dataset is **COCO** — *Common Objects in Context* — one of the best-known detection datasets. It ships object labels, bounding boxes, and full segmentation masks; this chapter uses only the boxes.

# Downloading COCO

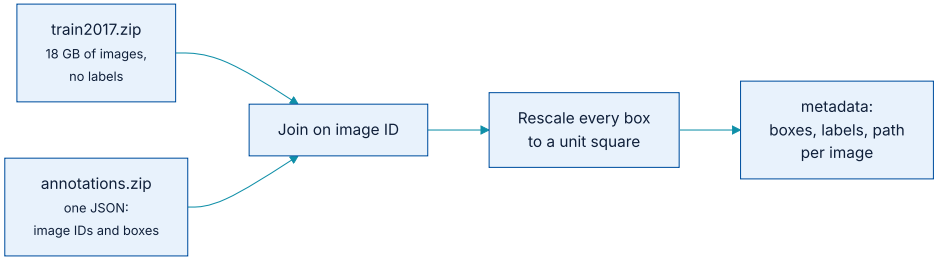
listing 12.1 — the 2017 COCO dataset PYTHON

```python
images_path = keras.utils.get_file(
    "coco",
    "http://images.cocodataset.org/zips/train2017.zip",
    extract=True,
)
annotations_path = keras.utils.get_file(
    "annotations",
    "http://images.cocodataset.org/annotations/annotations_trainval2017.zip",
    extract=True,
)
```

The first gives an **unlabelled directory of images**; the second gives all the metadata. They have to be joined.

SECTION 12.2.1

# Joining images to their boxes




COCO associates each image file with an ID, and each bounding box with one of those IDs.

# . . . and normalising the coordinates while you are at it

listing 12.2 — parsing the annotations PYTHON

```python
with open(f"{annotations_path}/annotations/instances_train2017.json") as f:
    annotations = json.load(f)
images = {image["id"]: image for image in annotations["images"]}

def scale_box(box, width, height):
    scale = 1.0 / max(width, height)          # longest side becomes 1.0
    x, y, w, h = [v * scale for v in box]
    x += (height - width) * scale / 2 if height > width else 0    # centre the
    y += (width - height) * scale / 2 if width > height else 0    # short side
    return [x, y, w, h]
```

The two centring lines matter: images are not square, so scaling by the longest side leaves letterboxing — and the boxes have to be **shifted by half that gap** or every prediction is offset.

# …and collating it per image

aggregating by image ID

PYTHON

```python
metadata = {}
for annotation in annotations["annotations"]:
    id = annotation["image_id"]
    metadata.setdefault(id, {"boxes": [], "labels": []})
    image = images[id]
    metadata[id]["boxes"].append(
        scale_box(annotation["bbox"], image["width"], image["height"]))
    metadata[id]["labels"].append(annotation["category_id"])
    metadata[id]["path"] = images_path + "/train2017/" + image["file_name"]

metadata = list(metadata.values())
```

The result is one record per image carrying **its boxes, its labels, and its file path** — which is what the target-array builder consumes.

SECTION 12.2.2

# The core idea: a grid of small predictors

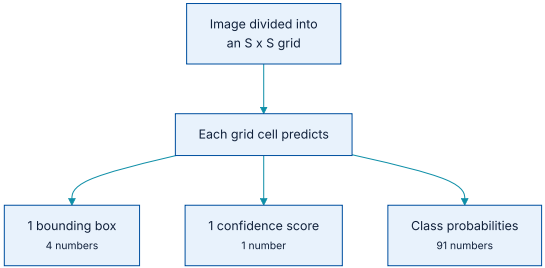



Figure 12.4 — each grid cell emits a box, a confidence, and a class distribution.

The original paper predicted several boxes per cell; this version keeps **one box per cell** for simplicity.

SECTION 12.2.2

# Why a confidence score is needed at all

So each cell also outputs a **confidence**: high where an object is detected, near zero where there is none. **Most cells should report near-zero confidence**, and that is the correct behaviour, not a failure.

# The backbone: ResNet, not Xception

listing 12.5 — loading ResNet PYTHON

```python
image_size = 448

backbone = keras_hub.models.Backbone.from_preset("resnet_50_imagenet")
preprocessor = keras_hub.layers.ImageConverter.from_preset(
    "resnet_50_imagenet",
    image_size=(image_size, image_size),
)
```

- **ResNet** is structurally similar to Xception but downsamples with **strides instead of pooling** — and chapter 11 established that strides preserve location, which detection needs.
- **448 × 448** rather than 180 × 180: **input size matters a great deal** for detection, because small objects vanish at low resolution.

# The detection head

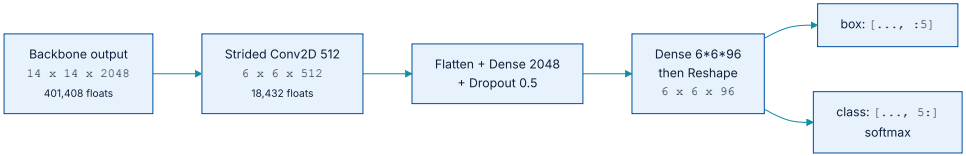
listing 12.6 — attaching the head PYTHON

```python
grid_size, num_labels = 6, 91

inputs = keras.Input(shape=(image_size, image_size, 3))
x = backbone(inputs)
x = layers.Conv2D(512, (3, 3), strides=(2, 2))(x)      # shrink the feature map
x = layers.Flatten()(x)
x = layers.Dense(2048, activation="relu", kernel_initializer="glorot_normal")(x)
x = layers.Dropout(0.5)(x)
x = layers.Dense(grid_size * grid_size * (num_labels + 5))(x)
x = layers.Reshape((grid_size, grid_size, num_labels + 5))(x)

box_predictions = x[..., :5]                           # 4 box numbers + confidence
class_predictions = layers.Activation("softmax")(x[..., 5:])
model = keras.Model(inputs, {"box": box_predictions, "class": class_predictions})
```

The `+ 5` is four box coordinates plus one confidence. Everything after that is the class distribution.

SECTION 12.2.2

# Following the shapes through the head




The strided convolution exists purely to get the float count down to something a dense layer can accept.

**401,408**
floats per image out of the backbone

**18,432**
after the strided convolution

SECTION 12.2.2

# One design choice worth noticing

> Because the **entire feature map is flattened** before the dense layers, every grid detector can see features from **the whole image**. There is **no locality constraint**

That is deliberate: **large objects will not stay contained inside a single grid cell**, so a cell responsible for one needs to see beyond its own square.

**77.8 M**
total parameters

**297 MB**
model size

SECTION 12.2.3

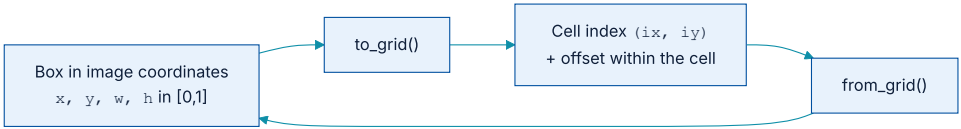
# Aligning the labels with the grid

> The assignment rule: **each grid detector is responsible for any box whose centre falls inside its cell**. A box straddling several cells still belongs to **exactly one of them**

This is why the box format matters: the coordinates are stored as centre-x, centre-y, width, height, so the cell assignment is a direct lookup.

SECTION 12.2.3

# Two coordinate systems, and the pair of functions between them




The model predicts positions relative to a cell; the labels arrive relative to the image.

**`x` and `y` are relative to the grid cell** (0 to 1 within it), while **`w` and `h` stay relative to the whole image**. The widths and heights need no conversion; **only the centres do**

# . . . written out

converting to and from the grid

PYTHON

```python
def to_grid(box):
    x, y, w, h = box
    cx, cy = (x + w / 2) * grid_size, (y + h / 2) * grid_size   # centre, in cells
    ix, iy = int(cx), int(cy)                                   # which cell
    return (ix, iy), (cx - ix, cy - iy, w, h)                   # offset within it

def from_grid(loc, box):
    (xi, yi), (x, y, w, h) = loc, box
    x = (xi + x) / grid_size - w / 2
    y = (yi + y) / grid_size - h / 2
    return (x, y, w, h)
```

Getting one of these subtly wrong is the classic detection bug: the loss still falls, the boxes still appear, and they are **consistently in the wrong place**

# Building the two target arrays

listing 12.7 — creating the targets PYTHON

```python
class_array = np.zeros((len(metadata), grid_size, grid_size))
box_array = np.zeros((len(metadata), grid_size, grid_size, 5))

for index, sample in enumerate(metadata):
    for box, label in zip(sample["boxes"], sample["labels"]):
        (x, y, w, h) = box
        left, right = math.floor(x * grid_size), math.ceil((x + w) * grid_size)
        bottom, top = math.floor(y * grid_size), math.ceil((y + h) * grid_size)
        class_array[index, bottom:top, left:right] = label     # every cell it covers
        loc, grid_box = to_grid(box)
        box_array[index, loc[1], loc[0]] = (*grid_box, 1.0)    # only the CENTRE cell
```

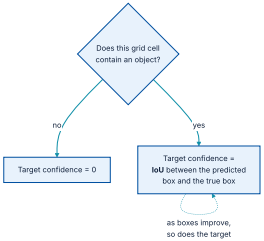
Note the asymmetry: **the class map marks every cell the box overlaps**, while **the box array marks only the cell containing its centre**. Overlapping boxes are **deliberately not handled**, to keep the example readable.

SECTION 12.2.4

# Two losses, summed

| OUTPUT | LOSS | NOTE |
|---|---|---|
| **class** | `sparse_categorical_crossentropy` | Entirely ordinary. |
| **box** | Sum-squared error, custom | Computed **only for grid cells that actually contain a box** in the labels. |

# The clever part: what confidence is trained against




The confidence target is not a flag — it is a measure of how good the box currently is.

The authors wanted confidence to reflect **not just the presence of an object but how good the predicted box is**. So as the model gets better at locating boxes, **the IoU rises and the learned confidence rises with it**

# Computing IoU for a batch of boxes

listing 12.10 — intersection area PYTHON

```python
from keras import ops

def unpack(box):
    return box[..., 0], box[..., 1], box[..., 2], box[..., 3]

def intersection(box1, box2):
    cx1, cy1, w1, h1 = unpack(box1)
    cx2, cy2, w2, h2 = unpack(box2)
    left   = ops.maximum(cx1 - w1 / 2, cx2 - w2 / 2)
    bottom = ops.maximum(cy1 - h1 / 2, cy2 - h2 / 2)
    right  = ops.minimum(cx1 + w1 / 2, cx2 + w2 / 2)
    top    = ops.minimum(cy1 + h1 / 2, cy2 + h2 / 2)
    return ops.maximum(0.0, right - left) * ops.maximum(0.0, top - bottom)
```

The `ops.maximum(0.0, ...)` at the end is what handles **boxes that do not overlap at all**: a negative width would otherwise produce **a spurious positive area**

# Running it, and reading the prediction

listing 12.13 — visualising a prediction PYTHON

```python
x, y = next(iter(val_dataset.rebatch(1)))
preds = model.predict(x)

boxes = preds["box"][0]
classes = np.argmax(preds["class"][0], axis=-1)      # most likely label per cell

draw_prediction(path, boxes, classes, cutoff=0.1)    # a LOW cutoff, deliberately
```

The cutoff is set low because, as the next slide says plainly, **the model is not a very good detector yet**

SECTION 12.2.4

# The chapter says so out loud

> Training takes **over an hour on a free Colab GPU**, and the model is **still undertrained** — the validation loss is still falling when it stops. It is starting to understand box locations and labels, **but it is not accurate**

This is worth dwelling on: the book publishes a result that does not work well, and explains why. **The purpose of the exercise is to feel how detection training behaves**, not to produce a usable COCO detector.

SECTION 12.2.4

# What would actually be needed

1. **Train for more epochs** — the loss had not converged.
2. **Use the whole COCO dataset**, not the subset.
3. **Data augmentation** — translating and rotating both the images **and their boxes**.
4. **Improve the class probability map** for overlapping boxes.
5. **Predict multiple boxes per grid location**, with a larger output grid.

> Together those approach the original YOLO training recipe. Doing it properly would take **a large amount of compute and time**, which is the honest reason to reach for a pretrained detector instead.

# 03

# A pretrained RetinaNet

The same principles, with one important architectural difference.

SECTION 12.3

# The difference that matters: scale


RetinaNet uses its ConvNet differently, to handle small and large objects simultaneously.

In our YOLO we used only the **final** backbone output. Those features map to large areas of the input, so they are **not very effective at finding small objects**

SECTION 12.3

# Loading it, and the box format argument

loading RetinaNet

PYTHON

```python
detector = keras_hub.models.ObjectDetector.from_preset(
    "retinanet_resnet50_fpn_v2_coco",
    bounding_box_format="rel_xywh",     # same format as our YOLO, so the same
)                                       # drawing utilities work unchanged

predictions = detector.predict(image)
```

`rel` means **relative to the image size** — coordinates in [0, 1]. Most Keras models and layers that handle boxes accept this argument, and **setting it wrongly produces boxes in the wrong place with no error at all**

# What comes back

inspecting the prediction

```python
[(k, v.shape) for k, v in predictions.items()]
# [("boxes", (1, 100, 4)),
#  ("confidence", (1, 100)),
#  ("labels", (1, 100)),
#  ("num_detections", (1,))]

predictions["boxes"][0][0]
```

OUTPUT

```
array([0.53, 0.00, 0.81, 0.29], dtype=float32)
```

`num_detections` is the one to read first — the arrays are padded to 100, so **the rest is not meaningful**.

# Drawing the detections

listing 12.15 — running inference PYTHON

```python
fig, ax = plt.subplots(dpi=300)
draw_image(ax, path)

num_detections = predictions["num_detections"][0]
for i in range(num_detections):
    box = predictions["boxes"][0][i]
    label = predictions["labels"][0][i]
    label_name = keras_hub.utils.coco_id_to_name(label)
    draw_box(ax, box, label_name, label_to_color(label))

plt.show()
```

That is the whole of using a pretrained detector: load, predict, draw.

SECTION 12.3

# The pointillist painting test

> RetinaNet generalises to **a pointillist painting** with ease, despite never having been trained on that style. Paintings and photographs differ enormously at the pixel level but **share a similar structure at a high level**

And the book draws an architectural conclusion from it: this is **an advantage of single-stage detectors**.

SECTION 12.3

# Why single-stage is more robust to novelty

**Two-stage**

Forced to classify **small patches in isolation** — which is extra difficult when a small patch of pixels looks nothing like the training data.

**Single-stage**

Can draw on features from **the entire input**, so it is more robust to novel test-time inputs.

Which connects back to chapter 5: robustness comes from **having more context to interpolate from**, not from having seen this exact style before.

SUMMARY

# What has to survive this chapter

1. **Detection when you do not need pixels** — it is cheaper to run and far cheaper to label than segmentation.
2. **Two-stage proposes then classifies**; **single-stage predicts boxes and labels jointly** and is faster.
3. **YOLO divides the image into a grid**, and each cell predicts a box, a confidence, and a class distribution.
4. **Confidence is trained against IoU**, not a 0/1 flag — so it measures *how good* the box is, not only whether something is there.
5. **Input resolution matters** for detection, far more than for classification.

SUMMARY

# . . . and the two things worth carrying further

**An honest negative result**

The from-scratch model **stays undertrained**, and the chapter says so. Training a real COCO detector takes serious compute — which is the argument for pretrained models, stated with evidence.

**Robustness has a structural cause**

RetinaNet handles a painting it never saw because it reads the **whole image**, and uses features from **several depths**.

NOTEBOOK

03_pretrained_retinanet.ipynb

NEXT

Chapter 13 — Timeseries forecasting